Avatar Generation Service — Updated Development Plan

HiDream-First Architecture with Claude Code Phases

Updated March 15, 2026

Executive Summary

This plan outlines a web service that takes a text prompt or image and outputs a complete, TalkingHead-ready GLB avatar with split lips, teeth, tongue, eyes, 52 ARKit blend shapes, 15 Oculus visemes, a Mixamo-compatible rig, and animations — all automatically. The plan uses a HiDream-first approach for concept image generation (MIT licensed, best quality, best prompt adherence), with FLUX Kontext as an optional enhancement for multi-view turnarounds.

---

1. Why This Is Feasible Now

Five open-source breakthroughs from 2025 collectively crack every stage of this pipeline:

HiDream-I1 (April 2025, MIT License) — 17B parameter text-to-image model with state-of-the-art quality. Sparse DiT with MoE architecture. Four text encoders (CLIP, T5, Llama 3.1) for fine-grained prompt control. Three variants: Full (50 steps, highest quality), Dev (28 steps, balanced), Fast (16 steps, real-time iteration). Achieves best-in-class scores on HPS v2.1, GenEval, and DPG benchmarks.

TRELLIS.2 (Microsoft, Dec 2025, MIT License) — 4B parameter image-to-3D model with PBR materials. O-Voxel representation handles open surfaces, non-manifold geometry, and transparency. Generation in 3–60 seconds depending on resolution. Full training code available for fine-tuning.

Hunyuan3D 2.1/2.5 (Tencent, 2025) — Text/image-to-3D with PBR textures. Multi-view input support (front/back/left). Shape generation on 6GB VRAM with mini/turbo models. Blender addon and ComfyUI integration. Hunyuan3D Studio includes humanoid auto-rigging research.

UniRig (VAST-AI/Tripo + Tsinghua, SIGGRAPH 2025, MIT License) — Automatic skeleton prediction and skinning weights for any 3D mesh. GPT-based autoregressive skeleton generation with Skeleton Tree Tokenization. 215% improvement over previous methods in rigging accuracy. Predicts Mixamo-compatible bone structures including spring bones. 1–5 seconds inference.

Deformation Transfer for ARKit (open source) — Sumner et al. algorithm transfers all 52 ARKit facial expressions from a reference mesh to any target mesh with different topology. Open

source Python implementation available. Combined with ICT FaceKit (free reference face with blend shapes), this automates the hardest step in the pipeline.

Market Context: Ready Player Me was acquired by Netflix in December 2025 and shut down all services on January 31, 2026. Over 8,000 developers need an alternative avatar system. No one has assembled these open-source tools into a unified service yet.

---

2. Refined Architecture: HiDream-First, Dual-Path

Why HiDream-I1 for Concept Generation

HiDream-I1 is the primary image generator because:

1. **MIT License** — fully commercial-friendly, unlike FLUX Kontext Dev (non-commercial license) or FLUX 2 Pro (paid API only). This is critical for a commercial service.
2. **Superior prompt adherence** — leads on GenEval and DPG benchmarks. The four-encoder architecture (OpenCLIP ViT-bigG, OpenAI CLIP ViT-L, T5-v1.1-xxl, Llama-3.1-8B-Instruct) allows sending structured character attributes to different encoders simultaneously.
3. **Style versatility** — state-of-the-art across photorealistic, cartoon, anime, and artistic styles on HPS v2.1.
4. **VRAM accessible** — FP8 version runs on 16GB VRAM, Q8 GGUF on 16GB with offloading (~7s/iteration). Full BF16 needs 27GB+.

Why FLUX Kontext as Optional Enhancement (Not Primary)

FLUX Kontext excels at maintaining character identity across edits — it outperforms HiDream-E1 on consistency benchmarks. The turnaround LoRA is specifically trained for generating multi-view character sheets. However:

- The Dev version has a non-commercial license
- Pro/Max are API-only at $0.05–0.07/image
- Not needed for the template path (just needs face close-up)
- Not strictly needed for AI path (TRELLIS.2/Hunyuan3D work from single images)

Use FLUX Kontext only as an optional quality boost for the AI mesh path, generating explicit multi-view inputs for Hunyuan3D Multi-View.

HiDream-E1 Role

HiDream-E1/E1.1 is the instruction-based image editing model. In this pipeline it can serve as:

- **Style transfer**: "Convert this character to anime style" or "Make this character look cyberpunk"
- **Iterative refinement**: User says "make the hair longer" or "add a scar" on the approved concept
- **Face close-up generation**: Zoom and enhance the face region from the full-body shot

However, it has a 768×768 resolution limit and community reports indicate inconsistency issues. Use it for creative transformation tasks, not for precision multi-view generation.

The Pipeline

```
User Input (text prompt or image)
        |
        ▼
┌──────────────────────────────────────────────────┐
|  PROMPT DECOMPOSITION (LLM — Claude API or local)|
|  Parse into: body type, style, face features,    |
|  hair, clothing, colors, special features        |
|  → Select template + build optimized prompts     |
|  → Route structured attributes to HiDream's      |
|    four text encoders                            |
└──────────────────────────────────────────────────┘
        |
        ▼
┌──────────────────────────────────────────────────┐
|  HERO IMAGE (HiDream-I1, MIT License)            |
|  Generate full-body T-pose character on white bg  |
|  HiDream-I1-Dev: 28 steps, ~7s on RTX 3080      |
|  HiDream-I1-Full Q8: 50 steps, ~15s             |
|  Resolution: 1024×1024                           |
|  User sees preview → approve / regenerate / edit |
└──────────────────────────────────────────────────┘
        |
        ▼ (User approves)
┌──────────────────────────────────────────────────┐
|  OPTIONAL: ITERATIVE REFINEMENT (HiDream-E1.1)   |
|  "Make hair longer" / "Add armor" / "Change style"|
|  Only if user requests changes to approved image  |
└──────────────────────────────────────────────────┘
        |
        ▼
┌──────────────────────────────────────────────────┐
```

```
│  FACE CLOSE-UP (HiDream-I1)                        │
│  Generate high-res face from same prompt +         │
│  reference. 1024×1024 or 2048×2048 for texture    │
│  detail. Eyes, lips, skin texture, features.      │
└──────────────────────────────────────────────────┘

        │
        ├──────────────── TEMPLATE PATH ────────────────┐
        │                                               │
        │  ┌──────────────────────────────────────────┐ │
        │  │  SELECT + BLEND TEMPLATE                 │ │
        │  │  Body type → choose from 3-5 base meshes  │ │
        │  │  Face shape → parametric shape key blending │ │
        │  │  Templates pre-built with:               │ │
        │  │  - Split lips, teeth, tongue, eyes       │ │
        │  │  - Proper face topology (~2000 faces/head) │ │
        │  │  - Pre-rigged Mixamo skeleton            │ │
        │  │  - 52 ARKit + 15 Oculus viseme shapes    │ │
        │  │  - Total: 8k-10k faces                   │ │
        │  └──────────────────────────────────────────┘ │
        │                       │                       │
        │  ┌──────────────────────────────────────────┐ │
        │  │  TEXTURE PROJECTION                      │ │
        │  │  Project face close-up onto template UV  │ │
        │  │  Body texture from HiDream-I1 body shot  │ │
        │  │  Blender headless: bake projected textures │ │
        │  │  OR: Hunyuan3D-Paint on template mesh    │ │
        │  └──────────────────────────────────────────┘ │
        │                       │                       │
        │                       ▼                       │
        │              MERGE + EXPORT ◀─────────────────┘
        │
        ├──────────────── AI MESH PATH ─────────────────┐
        │  (Optional enhancement: FLUX Kontext turnaround  │
        │   LoRA → multi-view → Hunyuan3D Multi-View)     │
        │                                               │
        │  ┌──────────────────────────────────────────┐ │
        │  │  3D MESH GENERATION                      │ │
        │  │  Feed hero image → TRELLIS.2 or Hunyuan3D  │ │
        │  │  TRELLIS.2: ~17s at 1024³ (MIT License)  │ │
        │  │  Hunyuan3D: ~30s with texture            │ │
        │  │  Output: raw mesh with PBR materials     │ │
        │  └──────────────────────────────────────────┘ │
        │                       │                       │
        │  ┌──────────────────────────────────────────┐ │
```

```
        |  |  RETOPOLOGY                                  | |
        |  |  Instant Meshes CLI → target 8k-10k faces    | |
        |  |  Preserve face density around eyes/mouth     | |
        |  |  Blender headless: cleanup + UV unwrap       | |
        |  └──────────────────────────────────────────────┘ |
        |                       |                           |
        |  ┌──────────────────────────────────────────────┐ |
        |  |  FACE COMPONENT SEPARATION                   | |
        |  |  MediaPipe landmarks on rendered views       | |
        |  |  Project 2D landmarks → 3D vertex positions | |
        |  |  Separate eye sockets from head mesh         | |
        |  |  Insert prefab teeth + tongue, scaled        | |
        |  |  Insert eye sphere meshes                    | |
        |  └──────────────────────────────────────────────┘ |
        |                       |                           |
        |  ┌──────────────────────────────────────────────┐ |
        |  |  AUTO-RIG (UniRig, MIT License)              | |
        |  |  Skeleton prediction: 1-5 seconds            | |
        |  |  Skinning weight prediction                  | |
        |  |  Remap bone names to Mixamo convention       | |
        |  └──────────────────────────────────────────────┘ |
        |                       |                           |
        |  ┌──────────────────────────────────────────────┐ |
        |  |  DEFORMATION TRANSFER                        | |
        |  |  NRICP: fit ICT FaceKit ref → target face    | |
        |  |  Transfer 52 ARKit blend shapes              | |
        |  |  Derive 15 Oculus visemes                    | |
        |  └──────────────────────────────────────────────┘ |
        |                       |                           |
        |                       ▼                           |
        |              MERGE + EXPORT ◀──────────────────────┘
                             |
                             ▼
┌──────────────────────────────────────────────────┐
|  ANIMATION ATTACHMENT                            |
|  Mixamo FBX library: idle, walk, run, wave, etc. |
|  Retarget to character skeleton via Blender      |
|  Bone name mapping: Mixamo → template → export   |
└──────────────────────────────────────────────────┘
                             |
                             ▼
┌──────────────────────────────────────────────────┐
|  VALIDATION + GLB EXPORT                         |
|  Automated checks:                               |
```

```
│  - All 67 morph targets present + correctly named│
│  - Mesh manifold, no inverted normals            │
│  - Skeleton has expected bone count              │
│  - Skinning weights sum to 1.0 per vertex        │
│  - Blend shapes deform within sane bounds        │
│  LOD generation: 10k, 5k, 3k variants           │
│  Draco compression option                        │
│  Export GLB with embedded morph targets           │
└──────────────────────────────────────────────────┘
                            │
                            ▼
┌──────────────────────────────────────────────────┐
│  TALKINGHEAD / HEADTTS / HEADAUDIO               │
│  Real-time lip-sync avatar in browser            │
│  GLB loaded via Three.js/WebGL                   │
│  Oculus visemes driven by HeadAudio or HeadTTS   │
│  ARKit expressions driven by emoji/mood system   │
│  Body animations from Mixamo FBX library         │
└──────────────────────────────────────────────────┘
```

---

3. Claude Code Development Phases

Phase 0: Foundation & Research Spike (1–2 weeks)

Goal: Prove each pipeline component works independently.

Claude Code tasks:

```
PROJECT SETUP:
- Initialize Python project with pyproject.toml
- Set up virtual environment with CUDA 12.4
- Create directory structure:
  avatar_service/
  ├── generators/      # Image generation (HiDream, FLUX)
  ├── mesh/            # 3D generation (TRELLIS.2, Hunyuan3D)
  ├── rigging/         # UniRig integration
  ├── blendshapes/     # Deformation transfer
  ├── blender_scripts/ # Headless Blender automation
  ├── templates/       # Template mesh library
  ├── animations/      # Mixamo FBX library
  ├── api/             # FastAPI backend
  ├── frontend/        # React web app
  └── tests/           # Automated validation

INSTALL DEPENDENCIES:
- Blender 4.x headless (apt install blender or portable)
- HiDream-I1-Dev (pip install diffusers, download from HF)
  - Text encoders: T5-v1.1-xxl, Llama-3.1-8B-Instruct
  - FP8 or Q8 GGUF for 16GB VRAM
- TRELLIS.2 (clone microsoft/TRELLIS.2, setup.sh)
  - Needs 24GB+ VRAM — test on cloud GPU if local won't fit
  - Alternative: Hunyuan3D-2mini-Turbo (fits 16GB)
- UniRig (clone VAST-AI-Research/UniRig)
  - Download skeleton + skinning checkpoints from HF
- Deformation transfer repo (clone vasiliskatr/deformation_transfer)
- ICT FaceKit reference mesh (download from ICT-VGL/ICT-FaceKit)
- Instant Meshes CLI (download binary from wjakob/instant-meshes)
- rembg (pip install rembg) for background removal

SMOKE TESTS (one script per component):
1. test_hidream.py:
   - Load HiDream-I1-Dev pipeline
   - Generate "full body T-pose fantasy warrior, white background"
   - Save output image, verify resolution and quality
   - Measure VRAM usage and generation time

2. test_trellis.py:
   - Load TRELLIS.2 pipeline (or Hunyuan3D-2mini)
   - Feed test image → generate 3D mesh
   - Export as GLB, verify mesh has vertices/faces
   - Measure VRAM and time
```

```
3. test_unirig.py:
   - Run generate_skeleton.sh on test GLB
   - Verify output FBX has skeleton hierarchy
   - Run generate_skin.sh for skinning weights
   - Run merge.sh to combine

4. test_deformation_transfer.py:
   - Load ICT FaceKit neutral + one blend shape
   - Load a test target face mesh
   - Run deformation transfer
   - Verify output mesh has deformed vertices

5. test_blender_headless.py:
   - blender --background --python script.py
   - Script: import GLB, list objects, export modified GLB
   - Verify Blender can manipulate meshes programmatically

6. test_talkinghead_glb.py:
   - Load a known-good RPM-compatible GLB
   - Verify it has expected morph target names
   - Use this as the reference format spec
```

Deliverable: Each component runs. You have benchmarks for VRAM, speed, and quality on your hardware. You understand every input/output format.

Phase 1: Template Mesh System (2–3 weeks)

Goal: Create parameterized template meshes with all TalkingHead requirements.

Claude Code tasks:

```
TEMPLATE CREATION (Blender Python scripts):
- Create 3-5 base body templates:
  1. Male realistic humanoid
  2. Female realistic humanoid
  3. Androgynous / neutral
  4. Stylized / cartoon (larger head, bigger eyes)
  5. Anime-style (optional, distinct proportions)

- Each template must have SEPARATE MESH OBJECTS for:
  - Body/head skin (main mesh)
  - Left eye sphere
  - Right eye sphere
  - Upper teeth
  - Lower teeth
  - Tongue
  - Eyelashes (geometry-based)
  - Eyebrows (geometry or cards)
  - Hair placeholder (cards or basic mesh)

- Face topology requirements:
  - ~1500-2000 faces for head region
  - Edge loops around eyes (concentric rings)
  - Edge loops around mouth (concentric rings)
  - Clean topology for blend shape deformation
  - Total body: 8000-10000 faces

- Each template fully UV-unwrapped:
  - Face gets largest UV island (~40% of UV space)
  - Body, arms, legs get remaining space
  - Clean UV seams at natural boundaries

PARAMETRIC VARIATION SYSTEM:
- Write Blender Python module: template_parametric.py
- Body shape keys on each template:
  - Height variation (tall/short)
  - Shoulder width (wide/narrow)
  - Hip width (wide/narrow)
  - Weight (thin/heavy)
  - Muscle tone (lean/muscular)
- Face shape keys:
  - Face width (narrow/wide)
  - Jaw shape (angular/round/pointed)
  - Forehead (high/low)
```

```
  - Cheekbones (prominent/subtle)
  - Nose width, nose length
  - Lip fullness
  - Eye size, eye spacing
- API: apply_body_params(template, params_dict) → modified mesh
- API: apply_face_params(template, params_dict) → modified mesh

PRE-RIG TEMPLATES:
- Each template has Mixamo-compatible skeleton already placed
- 65 bones matching Mixamo naming convention:
  Hips, Spine, Spine1, Spine2, Neck, Head,
  LeftShoulder, LeftArm, LeftForeArm, LeftHand,
  LeftHandThumb1-3, LeftHandIndex1-3, etc.
- Skinning weights pre-painted and tested
- Test: apply Mixamo idle animation → verify deformation looks correct

CREATE ARKIT BLEND SHAPES (via deformation transfer):
- Script: generate_arkit_shapes.py
- For each template:
  1. Load ICT FaceKit reference neutral face
  2. Load all 52 ICT FaceKit ARKit blend shapes
  3. Compute NRICP correspondence: reference → template face
  4. For each of 52 shapes, run deformation transfer
  5. Store as Blender shape keys with correct names:
     eyeBlinkLeft, eyeBlinkRight, jawOpen, jawForward,
     mouthSmile, mouthFrown, mouthPucker, mouthOpen,
     browDownLeft, browDownRight, browInnerUp,
     cheekPuff, cheekSquintLeft, noseSneerLeft, etc.
  6. Validate: each shape key deforms expected face region
     and doesn't produce inverted faces or wild displacements

CREATE OCULUS VISEME SHAPES:
- Script: generate_oculus_visemes.py
- Derive from ARKit mouth shapes:
  viseme_sil (silent/neutral)
  viseme_PP (bilabial: p, b, m)
  viseme_FF (labiodental: f, v)
  viseme_TH (dental: th)
  viseme_DD (alveolar: t, d, n)
  viseme_kk (velar: k, g)
  viseme_CH (postalveolar: ch, j, sh)
  viseme_SS (sibilant: s, z)
  viseme_nn (nasal: n, ng)
  viseme_RR (approximant: r)
```

```
  viseme_aa (open: a)
  viseme_E (front mid: e)
  viseme_I (front close: i)
  viseme_O (back mid: o)
  viseme_U (back close: u)
- Use TalkingHead's build-visemes-from-faceit-phonemes.py as reference
- Each viseme is a weighted combination of ARKit mouth shapes

ADDITIONAL MORPH TARGETS:
- mouthOpen, mouthSmile, eyesClosed, eyesLookUp, eyesLookDown
- These overlap with ARKit shapes but TalkingHead expects them
  with these specific names as well

VALIDATION:
- Script: validate_template.py
- Export each template as GLB with all morph targets
- Check: correct morph target count (52 + 15 + 5 = 72 total)
- Check: all morph target names match TalkingHead expectations
- Check: mesh face count within 8k-10k range
- Check: skeleton bone count and naming matches Mixamo
- Manual test: load GLB into TalkingHead test page,
  verify lip-sync, expressions, and body animation work
```

Deliverable: 3–5 template GLBs that work perfectly in TalkingHead. A parametric system that generates body/face variations.

Phase 2: AI Appearance Pipeline (2–3 weeks)

Goal: Given text or image input, generate unique appearance for template meshes.

Claude Code tasks:

```
PROMPT DECOMPOSITION:
- Module: prompt_decomposer.py
- Use Claude API (or local Llama 3.1) to parse user input:
  Input: "A cyberpunk female warrior with blue hair and scars"
  Output JSON:
  {
    "template": "female_realistic",
    "body_params": {"height": 0.6, "shoulders": 0.7, "muscle": 0.8},
    "face_params": {"jaw": "angular", "cheekbones": 0.7},
    "style": "realistic_scifi",
    "skin_tone": "medium",
    "hair": {"style": "short_asymmetric", "color": "electric_blue"},
    "features": ["facial_scar_left_cheek", "cybernetic_eye_right"],
    "clothing": "armored_bodysuit_dark",
    "color_palette": ["#1a1a2e", "#00d4ff", "#ff006e"]
  }
- Route structured attributes to HiDream's four encoders:
  - Encoder 1 (CLIP ViT-bigG): style tags, composition tags
  - Encoder 2 (CLIP ViT-L): visual feature tags
  - Encoder 3 (T5): natural language description
  - Encoder 4 (Llama): detailed instruction text

HERO IMAGE GENERATION:
- Module: hero_generator.py
- Load HiDream-I1-Dev (or Full Q8) pipeline
- Generate full-body T-pose on white background
- Prompt engineering for 3D-pipeline-friendly output:
  "Full body character, T-pose, arms slightly apart,
   facing camera directly, neutral expression, white background,
   even studio lighting, no shadows, clean silhouette,
   [character description from decomposer]"
- Generate 2-4 candidates, let user pick best
- Background removal with rembg
- Save as PNG with alpha channel

FACE CLOSE-UP GENERATION:
- Module: face_generator.py
- Generate separate high-res face-only image
- Same character identity, front-facing, neutral expression
- Higher detail: pores, iris color, lip texture, scars
- Resolution: 2048×2048 for texture quality
- This feeds directly into UV texture projection
```

```
ITERATIVE REFINEMENT (Optional):
- Module: refiner.py
- If user says "make hair longer" or "add tattoo":
  - Use HiDream-E1.1 for instruction-based editing
  - Input: approved hero image + edit instruction
  - Output: modified image maintaining character identity
  - Note: E1.1 limited to 768×768, upscale after if needed

TEXTURE PROJECTION ONTO TEMPLATE:
- Module: texture_projector.py (Blender headless)
- Input: face close-up PNG + body hero image PNG + selected template
- Process:
  1. Set up camera in Blender matching the generation view
  2. Project face image onto template's face UV island
  3. Project body image onto template's body UV islands
  4. Bake projected textures to 2048×2048 texture map
  5. Clean up seams with Blender's texture painting tools
  6. Generate normal map from texture for added detail
- Alternative: use Hunyuan3D-Paint to texture the template mesh
  - Feed template mesh + hero image to Paint model
  - Paint model generates multi-view consistent textures
  - Bake into UV map

HAIR SYSTEM:
- Module: hair_generator.py
- Option A: Hair cards (planes with alpha-textured hair)
  - Library of pre-made hair card meshes for common styles
  - Select based on prompt decomposer output
  - Apply hair color from character description
  - Parent to head bone
- Option B: Blender hair particle system → mesh conversion
  - More realistic but higher poly count
  - Reserve for high-LOD variant
- Hair mesh gets spring bones from UniRig for dynamic movement

FLUX KONTEXT TURNAROUND (Optional Enhancement):
- Module: turnaround_generator.py
- Only used if AI mesh path is selected AND user wants best quality
- Load FLUX Kontext Dev + Turnaround LoRA
- Input: approved hero image
- Output: 5-view turnaround sheet (front/¾/side/back/¾-back)
- Split sheet into individual views
- Feed into Hunyuan3D Multi-View for better 3D reconstruction
```

```
- Note: non-commercial license on Kontext Dev — only for
  non-commercial deployments, or use Kontext Pro API for commercial
```

Deliverable: Text/image → textured template mesh with unique appearance.

Phase 3: Pure AI Generation Path (2–3 weeks)

Goal: Support fully AI-generated novel meshes as alternative to templates.

Claude Code tasks:

```
3D MESH GENERATION:
- Module: mesh_generator.py
- Primary: TRELLIS.2 (MIT License, best topology)
  pipeline = Trellis2ImageTo3DPipeline.from_pretrained(
      'microsoft/TRELLIS.2-4B')
  mesh = pipeline.run(hero_image)[0]
- Fallback: Hunyuan3D-2mini-Turbo (fits 16GB VRAM)
- Also evaluate: PartCrafter (generates semantically separated parts)
- Output: raw mesh with PBR materials as GLB

AUTOMATIC RETOPOLOGY:
- Module: retopologizer.py
- Input: raw AI mesh (100k–600k triangles)
- Step 1: Instant Meshes CLI
  instant-meshes input.obj -o output.obj -f 10000 -D
  (target 10000 faces, deterministic)
- Step 2: Blender headless cleanup
  - Fix non-manifold edges
  - Recalculate normals
  - Remove floating vertices
  - Ensure face density is higher around head region
- Step 3: UV unwrap via Blender Smart UV Project
- Target: 8000–10000 faces total

FACE COMPONENT SEPARATION:
- Module: face_separator.py
- This is the hardest automated step. Approach:
  1. Render the retopologized mesh from front view in Blender
  2. Run MediaPipe FaceMesh on the rendered image
     → get 468 2D face landmarks
  3. Project 2D landmarks to 3D using Blender's ray casting
     → get 3D positions for eye centers, mouth corners,
       nose tip, jaw line, etc.
  4. Identify eye socket regions:
     - Find vertices within radius of eye landmark clusters
     - Separate these into eye socket holes
  5. Insert prefab eye spheres:
     - Scale to fit eye socket dimensions
     - Position at detected eye center 3D coordinates
     - Parent to Head bone
  6. Identify mouth region:
     - Vertices near mouth landmarks
     - Bisect mesh to create lip split (upper/lower lip edge)
```

```
  7. Insert prefab teeth and tongue:
     - Scale to fit mouth width/depth
     - Position based on jaw landmarks
     - Upper teeth: parent to Head bone
     - Lower teeth: parent to Jaw bone
     - Tongue: parent to Jaw bone
  8. Validate: render with wireframe, verify separations look correct
- FALLBACK: if automatic separation fails quality check,
  route to template path instead (hybrid fallback)

AUTO-RIG:
- Module: auto_rigger.py
- Run UniRig skeleton prediction:
  bash launch/inference/generate_skeleton.sh \
    --input retopo_mesh.glb --output skeleton.fbx
- Run UniRig skinning:
  bash launch/inference/generate_skin.sh \
    --input skeleton.fbx --output skinned.fbx
- Merge:
  bash launch/inference/merge.sh \
    --source skinned.fbx --target retopo_mesh.glb \
    --output rigged.glb
- Post-process in Blender:
  - Rename bones to Mixamo convention if needed
  - Verify bone hierarchy matches TalkingHead expectations
  - Clean up any extra bones UniRig may have added

DEFORMATION TRANSFER FOR BLEND SHAPES:
- Module: blendshape_generator.py
- This runs for BOTH paths (template and AI mesh)
- For AI mesh path, this is the critical step:
  1. Load ICT FaceKit reference neutral face + 52 ARKit shapes
  2. Compute NRICP correspondence between ICT ref and target face
     - Use landmark constraints from MediaPipe detection
     - Regularize to prevent fold-overs
  3. For each of 52 ARKit shapes:
     - Compute deformation field from ICT neutral → ICT shape
     - Apply deformation transfer to target mesh
     - Store as shape key in Blender
  4. Derive 15 Oculus visemes from ARKit mouth shapes
  5. Add extra targets: mouthOpen, mouthSmile, etc.
  6. Quality check: verify no inverted faces, reasonable displacement range

QUALITY SCORING:
```

```
- Module: quality_scorer.py
- Automated scoring for each generated avatar:
  - Blend shape quality: max displacement / average displacement ratio
  - Rig quality: test deformation with standard animation
  - Texture quality: check for UV seam artifacts
  - Overall mesh: manifold check, normal consistency
- If score below threshold → fall back to template path
- Return score to user as quality indicator
```

Deliverable: Image → complete rigged mesh with blend shapes (novel body shapes).

Phase 4: Animation & Export (1–2 weeks)

Claude Code tasks:

```
ANIMATION LIBRARY:
- Download standard Mixamo animations as FBX:
  idle_breathing, idle_looking, walk, run, jump,
  wave, nod, shake_head, clap, thinking, talking_gesture,
  sit, stand_to_sit, point, shrug, bow
- Store in animations/ directory
- Script: retarget_animation.py (Blender headless)
  - Load rigged character GLB
  - Import Mixamo FBX animation
  - Retarget: map Mixamo bones → character bones
  - Bake animation to character
  - Export as separate FBX files

GLB EXPORT PIPELINE:
- Module: exporter.py (Blender headless)
- Final export with ALL morph targets:
  - 52 ARKit shape keys (standard Apple naming)
  - 15 Oculus visemes (viseme_sil through viseme_U)
  - 5 extra (mouthOpen, mouthSmile, eyesClosed,
    eyesLookUp, eyesLookDown)
- Include embedded animations or reference separate FBX files
- LOD variants: export at 10k, 5k, 3k face counts
  - Use Blender Decimate modifier for lower LODs
  - Preserve shape keys through decimation
- Optional Draco mesh compression

TALKINGHEAD INTEGRATION TEST:
- Script: integration_test.py
- Spin up local TalkingHead test page
- Load generated GLB
- Automated checks via Puppeteer/Playwright:
  - Verify avatar renders without errors
  - Trigger each morph target, verify no console errors
  - Play idle animation, verify movement
  - Test lip-sync with HeadTTS sample audio
  - Screenshot each viseme for visual verification
```

Phase 5: Web Service & API (2-3 weeks)

Claude Code tasks:

```
BACKEND (Python FastAPI):
- POST /api/generate
  Body: { prompt: string, style: string, options: {} }
  Returns: { job_id: string }

- POST /api/generate-from-image
  Body: multipart form with image + options
  Returns: { job_id: string }

- GET /api/status/{job_id}
  Returns: { status: "generating_concept" | "user_approval" |
    "generating_mesh" | "rigging" | "blendshapes" |
    "exporting" | "complete" | "failed",
    progress: 0-100, preview_url: string }

- POST /api/approve/{job_id}
  Body: { approved: bool, selected_variant: int }

- POST /api/refine/{job_id}
  Body: { instruction: "make hair longer" }

- GET /api/download/{job_id}
  Returns: GLB file (or ZIP with GLB + FBX animations)

- WebSocket /ws/progress/{job_id}
  Real-time progress updates

JOB QUEUE:
- Redis + Celery workers
- GPU worker pool: separate queues for
  image_generation (HiDream), mesh_generation (TRELLIS.2),
  rigging (UniRig), blender_processing
- Priority queue for paid users
- Job timeout: 5 minutes max
- Retry logic with fallback to template path

FRONTEND (React + Three.js):
- Landing page with text prompt input
- Image upload with drag-and-drop
- Style selector: realistic / cartoon / anime / stylized
- Real-time preview using Three.js GLB viewer
- Customization panel: body sliders, face sliders
- Side-by-side comparison of generated variants
```

```
- TalkingHead live demo embed: test lip-sync in browser
- Download options: GLB, FBX, separate animations
- Gallery of community creations

CONTAINERIZATION:
- docker-compose.yml:
  - web: FastAPI + Uvicorn
  - worker-image: HiDream-I1 GPU worker
  - worker-3d: TRELLIS.2 / Hunyuan3D GPU worker
  - worker-rig: UniRig + Blender headless
  - redis: job queue broker
  - postgres: user accounts, job history
  - nginx: reverse proxy + static files
  - minio: S3-compatible object storage for GLB files
```

Phase 6: Optimization & Scale (Ongoing)

```
- FP8 quantization for all models to reduce VRAM
- Pre-generate template variations (cache popular body types)
- Batch processing: queue multiple avatars per GPU cycle
- CDN for serving generated GLB files (CloudFlare R2 or similar)
- Fine-tune HiDream-I1 on character art dataset for better T-pose output
- Fine-tune TRELLIS.2 on humanoid dataset for better body generation
- A/B test: template path quality vs AI mesh path quality
- User feedback loop: thumbs up/down on generated avatars
- Analytics: track which styles, body types, features are most popular
- Monitoring: GPU utilization, queue depth, error rates, generation time
```

---

4. Complete Technology Stack

All MIT Licensed (Commercial-Ready)

Component	Repo/Source	License	Role
HiDream-I1	HiDream-ai/HiDream-I1	MIT	Primary concept image generation
HiDream-E1.1	HiDream-ai/HiDream-E1	MIT	Iterative image editing/refinement

Component	Repo/Source	License	Role
TRELLIS.2	microsoft/TRELLIS.2	MIT	Image → 3D mesh generation
UniRig	VAST-AI-Research/UniRig	MIT	Automatic skeleton + skinning
Blender	blender.org	GPL	Headless mesh processing engine
ICT FaceKit	ICT-VGL/ICT-FaceKit	Free/Academic	Reference face with ARKit shapes
Deformation Transfer	vasiliskatr/deformation_transfer	Open	Transfer blend shapes to new meshes
Instant Meshes	wjakob/instant-meshes	BSD	Automatic retopology
TalkingHead	met4citizen/TalkingHead	MIT	Target platform / validation
HeadTTS	met4citizen/HeadTTS	MIT	TTS with viseme output
HeadAudio	met4citizen/HeadAudio	MIT	Audio-driven viseme detection
rembg	danielgatis/rembg	MIT	Background removal
MediaPipe	google/mediapipe	Apache 2.0	Face landmark detection

Optional (Non-Commercial or Paid)

Component	License	Role	When to Use
FLUX Kontext Dev	Non-commercial	Multi-view turnaround	Only if enhancing AI mesh path quality
FLUX Kontext Pro/Max	Paid API ($0.05–0.07/img)	Commercial turnaround	If need commercial multi-view turnarounds
Hunyuan3D 2.1	Tencent License	Alternative 3D generator	If TRELLIS.2 quality insufficient for specific cases
Hunyuan3D-Paint	Tencent License	Texture painting on meshes	Alternative to projection-based texturing

Infrastructure

Component	Choice	Role
Backend	Python FastAPI	REST API + WebSocket
Queue	Redis + Celery	Async GPU job processing
Database	PostgreSQL	Users, jobs, gallery
Object Storage	MinIO or CloudFlare R2	GLB file storage + CDN
Frontend	React + Three.js	Web UI + 3D preview
Containers	Docker Compose	Deployment

---

5. Hardware Requirements

Development (Your Current Machine: AMD + 32GB RAM + RTX 3080 16GB)

Task	Fits in 16GB?	Notes
HiDream-I1-Dev FP8	Yes (tight)	~16GB, may need offloading
HiDream-I1-Full Q8 GGUF	Yes	~18GB model, 7s/iteration with offload
HiDream-E1.1 Q8	Yes	Same architecture as I1
TRELLIS.2 shape only	Borderline	Needs 24GB for full; use cloud for this
Hunyuan3D-2mini-Turbo	Yes	Shape fits 6–9GB
UniRig inference	Yes	Moderate VRAM
Blender headless	Yes	CPU-bound, GPU for viewport only
Deformation transfer	Yes	CPU-bound

Verdict: You can develop and test the entire template path locally. For the AI mesh path, you'll need cloud GPU bursts for TRELLIS.2 full texture generation.

Recommended Upgrade

A used **RTX 4090 (24GB)** at $1,000–1,300 swapped into your existing system eliminates all constraints. Everything runs locally including TRELLIS.2 full pipeline.

Production Service

Scale	Hardware	Est. Monthly Cost
Demo/Hobby	Your RTX 3080 + cloud GPU bursts	$50–100
Small (50 avatars/day)	1× RTX 4090 server + web VPS	$200–400
Medium (500/day)	2× RTX 5090 + load balancer	$500–1,000
Large (5000/day)	Cloud GPU cluster (RunPod/Vast.ai)	$2,000–5,000

---

6. Timeline

Phase	Duration	Cumulative	Key Risk
Phase 0: Foundation	1–2 weeks	2 weeks	Dependency installation
Phase 1: Templates	2–3 weeks	5 weeks	Blend shape quality
Phase 2: AI Appearance	2–3 weeks	8 weeks	Texture projection quality
Phase 3: AI Mesh Path	2–3 weeks	11 weeks	Face component separation
Phase 4: Animation/Export	1–2 weeks	13 weeks	Bone name mapping
Phase 5: Web Service	2–3 weeks	16 weeks	Integration complexity
MVP (Template Path)	**~8 weeks**		Phases 0+1+2+4
Full Service	**~16 weeks**		All phases

The template-path MVP (Phases 0+1+2+4, skipping Phase 3) is achievable in ~2 months and produces a working service with guaranteed TalkingHead compatibility.

---

7. Risk Mitigation

Risk	Mitigation
Face separation fails on AI mesh	Fall back to template path automatically

Risk	Mitigation
Blend shape quality poor on extreme styles	Restrict to humanoid proportions; offer quality score
HiDream image quality insufficient for texture	Generate at 2048×2048; use Hunyuan3D-Paint as backup
VRAM constraints on RTX 3080	Use FP8/GGUF quantization; cloud GPU for heavy steps
TRELLIS.2 mesh topology too irregular	Pre-filter with retopology; reject if quality score low
TalkingHead morph target naming mismatch	Automated validation against reference GLB spec
Ready Player Me format changes in TalkingHead	Pin to known-good TalkingHead version; track upstream